



Power Query

The Complete Practical Guide

Power Query is Microsoft's powerful data transformation and ETL (Extract, Transform, Load) engine built into Excel, Power BI, and other Microsoft tools. This guide walks you through every concept with practical examples, diagrams, and real-world M language code.

[Data Import](#)[M Language](#)[Transformations](#)[Merging & Appending](#)[Query Folding](#)[ETL Pipeline](#)[Best Practices](#)**7+**

Topics Covered

40+

Transformations

M Code

Included

Diagrams

Visual Guide

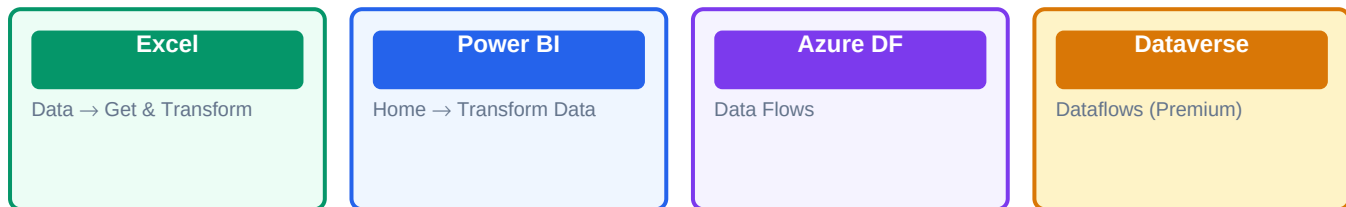
Contents at a Glance

01	What is Power Query?	Page 1
02	Power Query Architecture	Page 2
03	Data Import Sources	Page 3
04	M Language Deep Dive	Page 4
05	Transformation Steps	Page 5
06	Merge & Append Queries	Page 6
07	Query Folding & Best Practices	Page 7

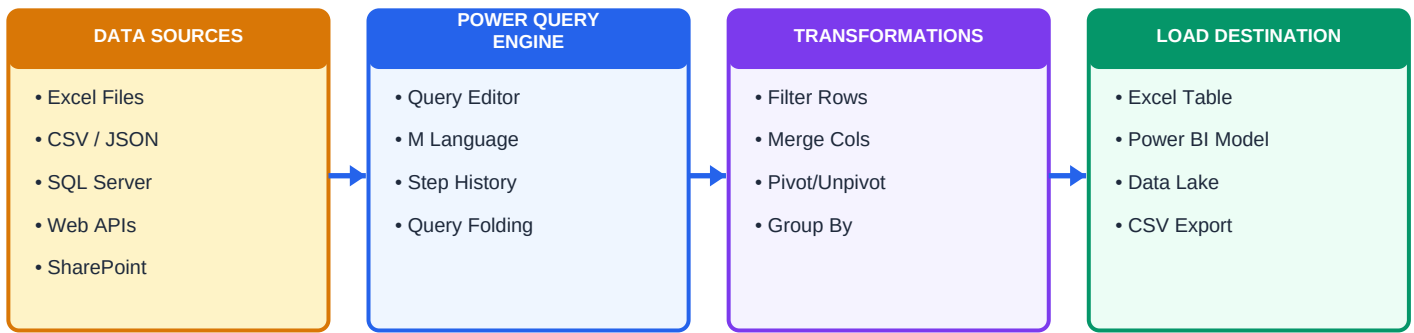
What is Power Query?

Power Query is a data connection technology that enables you to discover, connect, combine, and refine data across a wide variety of sources. Available in Excel (2016+), Power BI, and Azure Data Factory, it provides a visual, no-code interface backed by the M functional language.

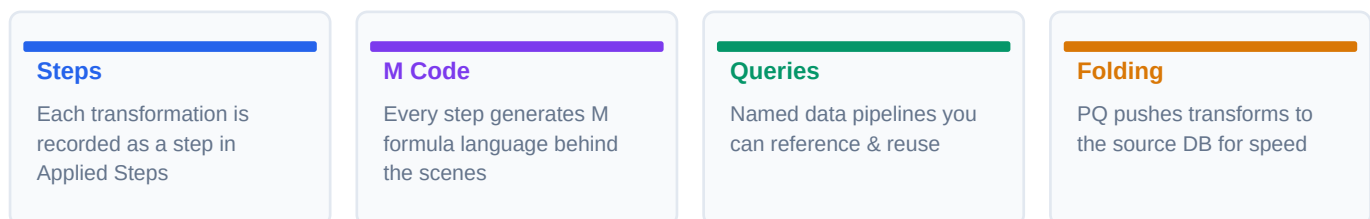
Where is Power Query Found?



Power Query Architecture



Key Concepts at a Glance



Data Import – Connecting to Sources

File Sources	Database Sources	Online / Cloud	Other Sources
Excel Workbook (.xlsx) Most common; supports named tables	SQL Server Native connector; supports folding	SharePoint List / Folder Microsoft 365 integration	Power BI Dataset Reuse existing models
CSV / Text file Delimiter auto-detected	MySQL / PostgreSQL Requires ODBC or native driver	Web Page (HTML tables) Scrape public web tables	Blank Query Write M code from scratch
JSON / XML Hierarchical data flattened automatically	Oracle / IBM DB2 Enterprise connectors	OData Feed REST-based data standard	Python / R Script Run scripts as data source
PDF Tables Extracts tables from PDF pages	Microsoft Access (.accdb) Local database files	Salesforce Objects CRM data direct import	ODBC / OLE DB Generic database connection
Folder Combine all files in a folder at once	Azure SQL Database Cloud-hosted SQL	Google Sheets (via URL) Public sheets as CSV	Azure Blob Storage Cloud file storage

Step-by-Step: Import from Excel

- 1 Open Excel → Data tab → Get Data → From File → From Workbook
- 2 Browse to the .xlsx file and click Import
- 3 Navigator window appears — select the Sheet or Table to load
- 4 Click Transform Data to open Power Query Editor
- 5 Apply transformations, then click Close & Load

M Code: Load Excel File

```
// Connect to Excel workbook
let
    Source = Excel.Workbook(File.Contents("C:\\Data\\Sales.xlsx"), null, true),
    Sales_Table = Source{[Item="SalesData",Kind="Table"]}[Data],
    #"Promoted Headers" = Table.PromoteHeaders(Sales_Table, [PromoteAllScalars=true]),
    #"Changed Type" = Table.TransformColumnTypes(#"Promoted Headers",
        {{ "Date", type date }, { "Amount", type number }})
in
    #"Changed Type"
```

M Language – Deep Dive

M (Mashup) is a functional, case-sensitive language that Power Query uses under the hood. Every click in the Query Editor generates an M expression. Understanding M lets you write custom transformations, dynamic parameters, and complex data logic.

M Syntax Fundamentals

let...in Structure

```
let
    Step1 = expression1,
    Step2 = expression2
in
    Step2
```

Records {}

```
[Name="Alice", Age=30]
```

Lists {}

```
{1, 2, 3, "hello", true}
```

Tables

```
Table.FromRecords(
    {[A=1, B="x"], [A=2, B="y"]})
```

Essential M Functions Reference

Category	Function	Description	Example
Table	Table.SelectRows	Filter rows by condition	<code>Table.SelectRows(t, each [Age]>18)</code>
Table	Table.AddColumn	Add computed column	<code>Table.AddColumn(t, "Full", [F]&" "&[L])</code>
Table	Table.Group	Group & aggregate	<code>Table.Group(t, "City", {"Cnt", each Table.RowCount(_)})</code>
Table	Table.Pivot	Pivot column values to headers	<code>Table.Pivot(t, vals, "Col", "Val")</code>
Text	Text.Upper / Lower	Change case	<code>Text.Upper([Name])</code>
Text	Text.Trim	Remove whitespace	<code>Text.Trim([City])</code>
Text	Text.Contains	Check substring	<code>Text.Contains([Email], "@")</code>
Date	Date.Year / Month / Day	Extract date parts	<code>Date.Year([OrderDate])</code>
List	List.Sum / Average	Aggregate list	<code>List.Sum({1, 2, 3})</code>
Number	Number.Round	Round number	<code>Number.Round([Price], 2)</code>

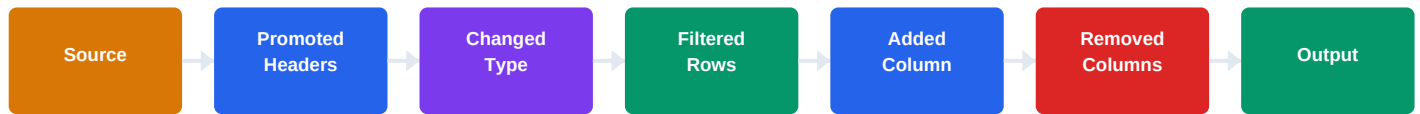
Error Handling in M

```
// Try-Otherwise pattern
SafeNum = try Number.FromText([RawValue]) otherwise 0,

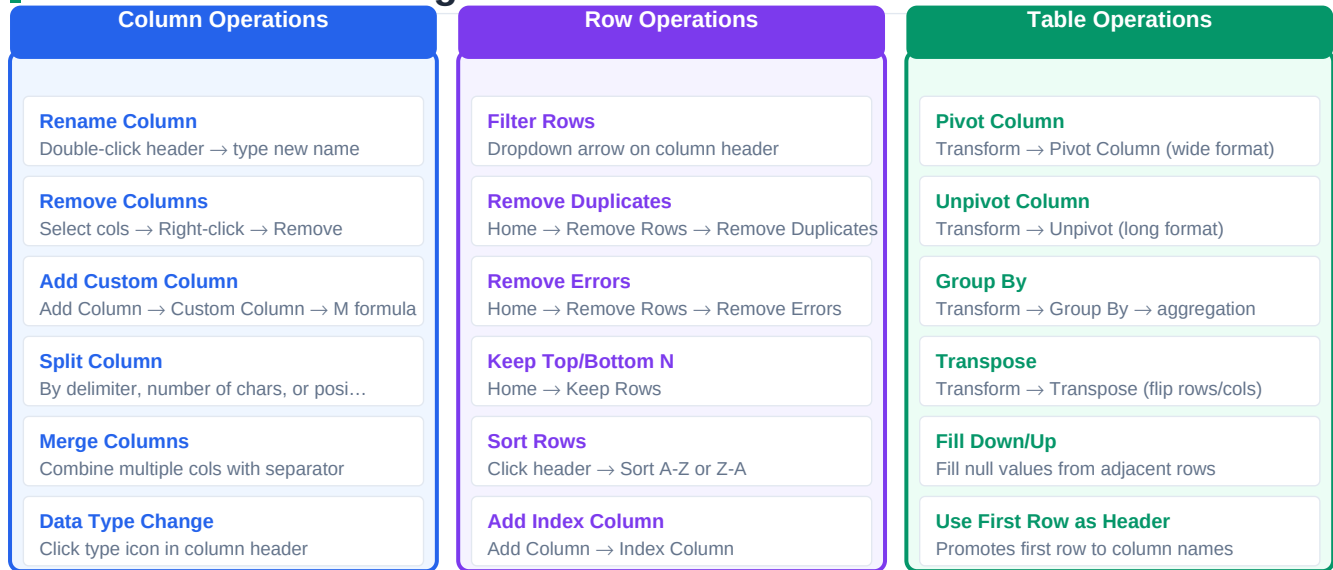
// Error replacement in whole column
CleanCol = Table.TransformColumns(Source,
    {"Price", each try _ otherwise null, type nullable number})
```

Transformation Steps – Visual Guide

Applied Steps Pipeline



Transformation Categories



Before & After: Pivot vs Unpivot

Before (Wide Format — Pivoted)

Student	Math	Science	English
Alice	85	92	78
Bob	76	88	91

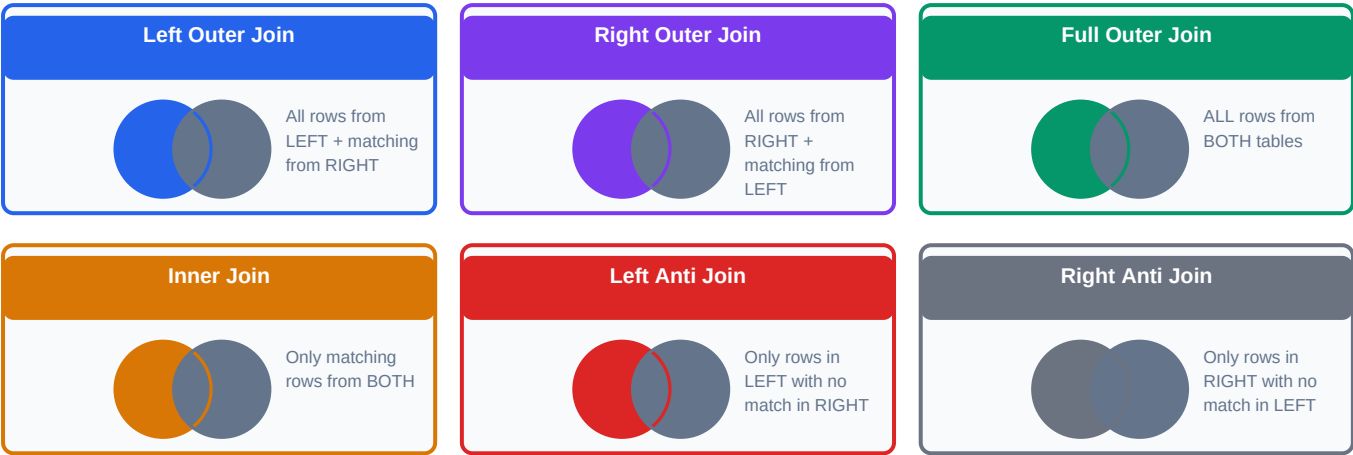
→
Unpivot

After (Long Format — Unpivoted)

Student	Subject	Score
Alice	Math	85
Alice	Science	92
Alice	English	78
Bob	Math	76

Merge & Append Queries

Merge (JOIN) Types – Visual Diagram



Append (UNION) Queries

Append stacks two or more queries on top of each other (like SQL UNION ALL). Columns are matched by name.

Query A (Jan)

OrderID	Product	Amount
1001	Widget	\$50
1002	Gadget	\$80

→
APPEND

Appended Result

OrderID	Product	Amount
1001	Widget	\$50
1002	Gadget	\$80
2001	Device	\$120
2002	Tool	\$35

Query B (Feb)

OrderID	Product	Amount
2001	Device	\$120

M Code: Merge Queries

```
// Merge Orders with Customers on CustomerID
let
    Source = Orders,
    Merged = Table.NestedJoin(Source, "CustomerID",
        Customers, "CustomerID", "CustomerData", JoinKind.LeftOuter),
    Expanded = Table.ExpandTableColumn(Merged, "CustomerData",
        {"Name","City"}, {"CustomerName","CustomerCity"})
in
    Expanded
```

Query Folding & Best Practices

What is Query Folding?

Query Folding is the ability of Power Query to translate your transformation steps back into the native query language of the data source (e.g., SQL). Instead of pulling all data into memory and processing it in PQ, the filtering/aggregation is done at the source — dramatically improving performance for large datasets.

Without Folding

1. ALL rows fetched from DB
2. All rows loaded into RAM
3. PQ filters/aggregates in memory
4. Slow for millions of rows

With Query Folding

1. PQ translates steps to SQL
2. DB executes the SQL query
3. Only relevant rows returned
4. Fast — DB optimizes the query

Tip: Right-click any Applied Step → View Native Query (greyed out = folding broke at that step)

Best Practices

1 Filter Early

Apply row filters as the first step to minimize data loaded into memory.

2 Rename for Clarity

Give each Applied Step a meaningful name (right-click → Rename).

3 Disable Auto-Refresh

Prevent background refresh during development (Query Properties).

4 Use Parameters

Create query parameters for file paths, dates, server names for reusability.

5 Avoid UI-Only Connectors

Prefer native DB connectors over ODBC where possible for better folding.

6 Document with Comments

Use // comments in M code for complex logic clarity.

Quick Reference Cheat Sheet

Shortcut	Action
Ctrl+T	Open Power Query Editor (Excel)
Right-click step	Rename / Delete / Move step
Ctrl+Z	Undo last transformation
M: each _	Lambda shorthand for row-level function
M: #shared	Browse all available M functions
M: Value.NativeQuery	Force/check query folding manually